

Udiddit, a social news aggregator

Introduction

Udiddit, a social news aggregation, web content rating, and discussion website, is currently using a risky and unreliable Postgres database schema to store the forum posts, discussions, and votes made by their users about different topics.

The schema allows posts to be created by registered users on certain topics, and can include a URL or a text content. It also allows registered users to cast an upvote (like) or downvote (dislike) for any forum post that has been created. In addition to this, the schema also allows registered users to add comments on posts.

Here is the DDL used to create the schema:

```
CREATE TABLE bad_posts (  
    id SERIAL PRIMARY KEY,  
    topic VARCHAR(50),  
    username VARCHAR(50),  
    title VARCHAR(150),  
    url VARCHAR(4000) DEFAULT NULL,  
    text_content TEXT DEFAULT NULL,  
    upvotes TEXT,  
    downvotes TEXT  
);  
  
CREATE TABLE bad_comments (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    post_id BIGINT,  
    text_content TEXT  
);
```

Part I: Investigate the existing schema

As a first step, investigate this schema and some of the sample data in the project's SQL workspace. Then, in your own words, outline three (3) specific things that could be improved about this schema. Don't hesitate to outline more if you want to stand out!

Violation of First Normal Form (1NF)

The `upvotes` and `downvotes` columns in `bad_posts` are stored as TEXT fields, likely containing comma-separated lists of usernames. This violates First Normal Form because it stores multiple values in a single column. This design makes it impossible to efficiently query vote counts, validate unique votes per user, or maintain referential integrity. Vote data should be stored in a separate normalized table with one row per vote.

Missing Users Table and Referential Integrity

The schema stores usernames as VARCHAR fields directly in both tables without any foreign key constraints. There's no dedicated users table to ensure username uniqueness, manage user registration, or enforce referential integrity. If a username changes or is deleted, there's no way to cascade updates across posts, comments, and votes, leading to orphaned data and inconsistencies.

Lack of Topics Normalization

Topics are stored as VARCHAR(50) directly in the `bad_posts` table, allowing duplicate topic names with inconsistent spelling or casing (e.g., "Technology" vs "technology"). This creates data redundancy and makes it difficult to enforce topic integrity, list all available topics, or add topic metadata like descriptions or moderators. Topics should be in a separate table with a foreign key relationship.

No Vote Type Separation

The current schema stores upvotes and downvotes in separate TEXT columns, but best practice is to use a single votes table with a `vote_type` column containing values like -1 (downvote), 0 (neutral), or 1 (upvote). This allows efficient calculation of total scores using SUM operations and prevents the same user from casting multiple votes on the same post. The current design has no mechanism to prevent duplicate votes or enforce one-vote-per-user-per-post rules.

Missing Constraints and Data Validation

Current schema lacks:

1. NOT NULL constraints on critical fields (username, title, post_id)
2. CHECK constraints to ensure posts have either URL or text_content (not both or neither)
3. UNIQUE constraints to prevent duplicate usernames

4. Foreign key constraints for referential integrity
5. Timestamps for created_at/updated_at tracking

Part II: Create the DDL for your new schema

Having done this initial investigation and assessment, your next goal is to dive deep into the heart of the problem and create a new schema for Udiddit. Your new schema should at least reflect fixes to the shortcomings you pointed to in the previous exercise. To help you create the new schema, a few guidelines are provided to you:

1. Guideline #1: here is a list of features and specifications that Udiddit needs in order to support its website and administrative interface:
 - a. Allow new users to register:
 - i. Each username has to be unique
 - ii. Usernames can be composed of at most 25 characters
 - iii. Usernames can't be empty
 - iv. We won't worry about user passwords for this project
 - b. Allow registered users to create new topics:
 - i. Topic names have to be unique.
 - ii. The topic's name is at most 30 characters
 - iii. The topic's name can't be empty
 - iv. Topics can have an optional description of at most 500 characters.
 - c. Allow registered users to create new posts on existing topics:
 - i. Posts have a required title of at most 100 characters
 - ii. The title of a post can't be empty.
 - iii. Posts should contain either a URL or a text content, **but not both**.
 - iv. If a topic gets deleted, all the posts associated with it should be automatically deleted too.
 - v. If the user who created the post gets deleted, then the post will remain, but it will become dissociated from that user.
 - d. Allow registered users to comment on existing posts:
 - i. A comment's text content can't be empty.
 - ii. Contrary to the current linear comments, the new structure should allow comment threads at arbitrary levels.
 - iii. If a post gets deleted, all comments associated with it should be automatically deleted too.
 - iv. If the user who created the comment gets deleted, then the comment will remain, but it will become dissociated from that user.
 - v. If a comment gets deleted, then all its descendants in the thread structure should be automatically deleted too.
 - e. Make sure that a given user can only vote once on a given post:
 - i. Hint: you can store the (up/down) value of the vote as the values 1 and -1 respectively.
 - ii. If the user who cast a vote gets deleted, then all their votes will remain, but will become dissociated from the user.

- iii. If a post gets deleted, then all the votes for that post should be automatically deleted too.
2. Guideline #2: here is a list of queries that Uddidit needs in order to support its website and administrative interface. Note that you don't need to produce the DQL for those queries: they are only provided to guide the design of your new database schema.
- a. List all users who haven't logged in in the last year.
 - b. List all users who haven't created any post.
 - c. Find a user by their username.
 - d. List all topics that don't have any posts.
 - e. Find a topic by its name.
 - f. List the latest 20 posts for a given topic.
 - g. List the latest 20 posts made by a given user.
 - h. Find all posts that link to a specific URL, for moderation purposes.
 - i. List all the top-level comments (those that don't have a parent comment) for a given post.
 - j. List all the direct children of a parent comment.
 - k. List the latest 20 comments made by a given user.
 - l. Compute the score of a post, defined as the difference between the number of upvotes and the number of downvotes
3. Guideline #3: you'll need to use normalization, various constraints, as well as indexes in your new database schema. You should use named constraints and indexes to make your schema cleaner.
4. Guideline #4: your new database schema will be composed of five (5) tables that should have an auto-incrementing id as their primary key.

Once you've taken the time to think about your new schema, write the DDL for it in the space provided here:

```
-- Table 1: Users table for user registration and authentication
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(25) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  last_login TIMESTAMP,
  CONSTRAINT users_username_unique UNIQUE (username),
  CONSTRAINT users_username_not_empty CHECK (LENGTH(TRIM(username)) > 0)
);
```

```

-- Index for finding users by filtering by last login
CREATE INDEX idx_users_last_login ON users (last_login);

-- Table 2: Topics table for organizing posts
CREATE TABLE topics (
    id SERIAL PRIMARY KEY,
    name VARCHAR(30) NOT NULL,
    description VARCHAR(500),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CONSTRAINT topics_name_unique UNIQUE (name),
    CONSTRAINT topics_name_not_empty CHECK (LENGTH(TRIM(name)) > 0)
);

-- Table 3: Posts table for user-created content
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    url VARCHAR(4000),
    text_content TEXT,
    topic_id INTEGER NOT NULL,
    user_id INTEGER,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CONSTRAINT posts_title_not_empty CHECK (LENGTH(TRIM(title)) > 0),
    CONSTRAINT posts_url_or_text_content CHECK (
        (url IS NOT NULL AND text_content IS NULL) OR
        (url IS NULL AND text_content IS NOT NULL)
    ),
    CONSTRAINT posts_topic_fk FOREIGN KEY (topic_id)
        REFERENCES topics (id) ON DELETE CASCADE,
    CONSTRAINT posts_user_fk FOREIGN KEY (user_id)
        REFERENCES users (id) ON DELETE SET NULL
);

-- Indexes for querying posts by topic, user, URL, and chronological order
CREATE INDEX idx_posts_topic_id ON posts (topic_id);
CREATE INDEX idx_posts_user_id ON posts (user_id);
CREATE INDEX idx_posts_url ON posts (url);
CREATE INDEX idx_posts_created_at_desc ON posts (created_at DESC);
CREATE INDEX idx_posts_topic_created ON posts (topic_id, created_at DESC);
CREATE INDEX idx_posts_user_created ON posts (user_id, created_at DESC);

```

```

-- Table 4: Comments table for threaded discussions
CREATE TABLE comments (
    id SERIAL PRIMARY KEY,
    text_content TEXT NOT NULL,
    post_id INTEGER NOT NULL,
    user_id INTEGER,
    parent_comment_id INTEGER,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CONSTRAINT comments_text_not_empty CHECK (LENGTH(TRIM(text_content)) > 0),
    CONSTRAINT comments_post_fk FOREIGN KEY (post_id)
        REFERENCES posts (id) ON DELETE CASCADE,
    CONSTRAINT comments_user_fk FOREIGN KEY (user_id)
        REFERENCES users (id) ON DELETE SET NULL,
    CONSTRAINT comments_parent_fk FOREIGN KEY (parent_comment_id)
        REFERENCES comments (id) ON DELETE CASCADE
);

-- Indexes for querying comments by post, parent, user, and chronological order
CREATE INDEX idx_comments_post_id ON comments (post_id);
CREATE INDEX idx_comments_parent_id ON comments (parent_comment_id);
CREATE INDEX idx_comments_user_id ON comments (user_id);
CREATE INDEX idx_comments_created_at_desc ON comments (created_at DESC);
CREATE INDEX idx_comments_user_created ON comments (user_id, created_at DESC);

-- Table 5: Votes table for upvotes and downvotes
CREATE TABLE votes (
    id SERIAL PRIMARY KEY,
    user_id INTEGER,
    post_id INTEGER NOT NULL,
    vote_value INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CONSTRAINT votes_value_check CHECK (vote_value IN (-1, 1)),
    CONSTRAINT votes_post_fk FOREIGN KEY (post_id)
        REFERENCES posts (id) ON DELETE CASCADE,
    CONSTRAINT votes_user_fk FOREIGN KEY (user_id)
        REFERENCES users (id) ON DELETE SET NULL,
    CONSTRAINT votes_user_post_unique UNIQUE (user_id, post_id)
);

-- Indexes for calculating post scores and querying votes
CREATE INDEX idx_votes_post_id ON votes (post_id);
CREATE INDEX idx_votes_user_id ON votes (user_id);

```

Part III: Migrate the provided data

Now that your new schema is created, it's time to migrate the data from the provided schema in the project's SQL Workspace to your own schema. This will allow you to review some DML and DQL concepts, as you'll be using INSERT...SELECT queries to do so. Here are a few guidelines to help you in this process:

1. Topic descriptions can all be empty
2. Since the bad_comments table doesn't have the threading feature, you can migrate all comments as top-level comments, i.e. without a parent
3. You can use the Postgres string function **regexp_split_to_table** to unwind the comma-separated votes values into separate rows
4. Don't forget that some users only vote or comment, and haven't created any posts. You'll have to create those users too.
5. The order of your migrations matter! For example, since posts depend on users and topics, you'll have to migrate the latter first.
6. Tip: You can start by running only SELECTs to fine-tune your queries, and use a LIMIT to avoid large data sets. Once you know you have the correct query, you can then run your full INSERT...SELECT query.
7. **NOTE:** The data in your SQL Workspace contains thousands of posts and comments. The DML queries may take at least 10-15 seconds to run.

Write the DML to migrate the current data in bad_posts and bad_comments to your new database schema:

```
-- MIGRATION STEP 1: Migrate all unique users from multiple sources
-- Users can exist in bad_posts (as post creators), bad_comments (as
commenters),
-- and in the upvotes/downvotes columns (as voters)

INSERT INTO users (username)
SELECT DISTINCT username
FROM (
    -- Users who created posts
    SELECT DISTINCT username
    FROM bad_posts
    WHERE username IS NOT NULL AND TRIM(username) != ''
    UNION
    -- Users who created comments
    SELECT DISTINCT username
```



```

FROM bad_comments
WHERE username IS NOT NULL AND TRIM(username) != ''

UNION
-- Users who cast upvotes
SELECT DISTINCT REGEXP_SPLIT_TO_TABLE(upvotes, ',') AS username
FROM bad_posts
WHERE upvotes IS NOT NULL AND upvotes != ''
UNION
-- Users who cast downvotes
SELECT DISTINCT REGEXP_SPLIT_TO_TABLE(downvotes, ',') AS username
FROM bad_posts
WHERE downvotes IS NOT NULL AND downvotes != ''
) AS all_usernames
WHERE username IS NOT NULL AND TRIM(username) != '';

-- MIGRATION STEP 2: Migrate all unique topics
-- Extract unique topics from bad_posts and create topic records with empty
descriptions

INSERT INTO topics (name)
SELECT DISTINCT topic
FROM bad_posts
WHERE topic IS NOT NULL AND TRIM(topic) != '';

-- MIGRATION STEP 3: Migrate posts
-- Join with users and topics tables to get proper foreign key references
-- Handle the url and text_content fields appropriately
INSERT INTO posts (id, title, url, text_content, topic_id, user_id)
SELECT
    bp.id,
    LEFT(bp.title, 100) AS title, -- Ensure title doesn't exceed 100
characters
    bp.url,
    bp.text_content,
    t.id AS topic_id,
    u.id AS user_id
FROM bad_posts bp
JOIN topics t ON t.name = bp.topic
LEFT JOIN users u ON u.username = bp.username; -- LEFT JOIN in case
username is NULL

```

```
-- MIGRATION STEP 4: Migrate comments as top-level comments
-- All comments will have parent_comment_id = NULL since threading doesn't
exist in old schema
```

```
INSERT INTO comments (id, text_content, post_id, user_id,
parent_comment_id)
SELECT
    bc.id,
    bc.text_content,
    bc.post_id,
    u.id AS user_id,
    NULL AS parent_comment_id -- All comments are top-level
FROM bad_comments bc
LEFT JOIN users u ON u.username = bc.username; -- LEFT JOIN in case
username is NULL
```

```
-- MIGRATION STEP 5: Migrate upvotes
-- Unwind comma-separated upvotes into individual vote records with
vote_value = 1
```

```
INSERT INTO votes (user_id, post_id, vote_value)
SELECT
    u.id AS user_id,
    upvotes_data.post_id,
    1 AS vote_value
FROM (
    SELECT
        id AS post_id,
        REGEXP_SPLIT_TO_TABLE(upvotes, ',') AS username
    FROM bad_posts
    WHERE upvotes IS NOT NULL AND upvotes != ''
) AS upvotes_data
JOIN users u ON u.username = upvotes_data.username;
```

```

-- MIGRATION STEP 6: Migrate downvotes
-- Unwind comma-separated downvotes into individual vote records with
vote_value = -1

INSERT INTO votes (user_id, post_id, vote_value)
SELECT
    u.id AS user_id,
    downvotes_data.post_id,
    -1 AS vote_value
FROM (
    SELECT
        id AS post_id,
        REGEXP_SPLIT_TO_TABLE(downvotes, ',') AS username
    FROM bad_posts
    WHERE downvotes IS NOT NULL AND downvotes != ''
) AS downvotes_data
JOIN users u ON u.username = downvotes_data.username;

-- Reset sequences to avoid ID conflicts for future inserts
-- This ensures new posts/comments will have IDs higher than migrated ones

SELECT SETVAL('posts_id_seq', (SELECT MAX(id) FROM posts));
SELECT SETVAL('comments_id_seq', (SELECT MAX(id) FROM comments));

```